

```
[3]: cuadrados = [x^2 | x <- [1..10]]
cuadrados
[1,4,9,16,25,36,49,64,81,100]

[5]: divisibles3 = [x | x <- [1..30], x `mod` 3 == 0]
divisibles3
[3,6,9,12,15,18,21,24,27,30]

[6]: parejas = [(x,y) | x <- [1..5], y <- [1..5], x < y]
parejas
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

[8]: divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n `mod` x == 0]
divisores 54
[1,2,3,6,9,18,27,54]

[19]: divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], n `mod` x == 0]
divisoresPropios 6
[1,2,3]

[20]: esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n
esPerfecto 6
True

[21]: esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n
esPerfecto 12
False

[22]: sumalista :: [Int] -> Int
sumalista xs = sum xs
sumalista [2,2,3,4,5]

Eta reduce
Found: Why Not:
sumalista xs = sum xs
sumalista = sum
15

[23]: mayoresQue10 :: [Int] -> Int
mayoresQue10 xs = length (filter (>10) xs)
mayoresQue10 [2,2,3,4,5]
0

[25]: productolista :: [Int] -> Int
productolista xs = foldl (*) 1 xs
productolista [2,2,3,4,5]

Eta reduce
Found: Why Not:
productolista xs = foldl (*) 1 xs
productolista = foldl (*) 1
Use product
Found: Why Not:
foldl (*) 1 product
120

[24]: celsiusAFahrenheit :: [Double] -> [Double]
celsiusAFahrenheit = map (\c -> c * 9/5 + 32)
celsiusAFahrenheit [0,30,60]
[32.0,86.0,140.0]

[26]: ajustarImpares :: [Int] -> [Int]
ajustarImpares = map (\x -> if odd x then x + 5 else x)
ajustarImpares [1,2,3,4,5]
[6,2,8,4,10]

[27]: nombresLargos :: [String] -> [String]
nombresLargos = filter (\nombre -> length nombre > 5)
nombresLargos ["Sarai","Uriel","Adriana","Margarita"]
["Adriana","Margarita"]

[29]: soloNegativos :: [Int] -> [Int]
soloNegativos = filter (<0)
soloNegativos [-1,2,-3,4,-5]
[-1,-3,-5]

[30]: edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios = filter (\edad -> edad >= 18 && edad <= 25)
edadesUniversitarios [21,2,13,34,15,19,22,81]
[21,15,19,22]

[47]: agregarIVA xs = map (\x -> x * 1.19) (filter (>=1000) xs)
agregarIVA[200000,1000,1500,8100]
[238000.0,1190.0,1785.0,9639.0]

Avoid lambda using 'infix'
Found: Why Not:
(\ x -> x * 1.19)(* 1.19)
[238000.0,1190.0,1785.0,9639.0]

[54]: import Data.Char (toLower)
longitudesVocales :: [String] -> [Int]
longitudesVocales palabras = [length p | p <- palabras, not (null p), toLower (head p) `elem` "aeiouáéíóú"]
longitudesVocales ["Sarai","Uriel","Adriana","Ivan","Margarita"]
[5,7,4]

[56]: cuadradosPares :: [Int] -> [Int]
cuadradosPares xs = map (^2) (filter even xs)
cuadradosPares [1,2,3,4,5,6,7,8,9,10]
[4,16,36,64,100]
```

```
[57]: hayNegativo :: [Int] -> Bool
      hayNegativo = any (<0)
      hayNegativo [-1,2,-3,4,-5]

      True
```

```
[58]: todosPares :: [Int] -> Bool
      todosPares = all even
      todosPares [1,2,3,4,5]

      False
```

```
[59]: todosPares :: [Int] -> Bool
      todosPares = all even
      todosPares [2,4,6]

      True
```

```
[61]: import Data.Char (isUpper)
      nombresCorrectos :: [String] -> Bool
      nombresCorrectos xs =
        all (\nombre -> not (null nombre) && isUpper (head nombre)) xs
      nombresCorrectos ["Sarai","Uriel","Nevo"]
```

Eta reduce

Found:

nombresCorrectos xs

= all (\ nombre -> not (null nombre) && isUpper (head nombre)) xs

... True

Why Not:

nombresCorrectos

= all (\ nombre -> not (null nombre) && isUpper (head nombre))

```
[73]: menoresQue100 :: [Int] -> [Int]
      menoresQue100 = takeWhile (<100)
      menoresQue100 [10,35,101,790,120,1000,180]

      [10,35]
```

```
[75]: sinCerosIniciales :: [Int] -> [Int]
      sinCerosIniciales = dropWhile (==0)
      sinCerosIniciales [0,0,465,745,0,2]

      [465,745,0,2]
```

```
[1]: procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
      procesarLecturas funcion lista = map funcion lista
```

Eta reduce

Found:

procesarLecturas funcion lista = map funcion lista

Why Not:

```
[2]: celsiusAFahrenheit :: Double -> Double
      celsiusAFahrenheit c = (c * 9 / 5) + 32
```

```
[3]: procesarLecturas celsiusAFahrenheit [0,12,20]

      [32.0,53.6,68.0]
```

```
[4]: crearFiltro :: (a -> Bool) -> [a] -> [a]
      crearFiltro condicion lista = filter condicion lista
```

Eta reduce

Found:

crearFiltro condicion lista = filter condicion lista

Why Not:

```
[5]: soloPares :: [Int] -> [Int]
      soloPares lista = crearFiltro even lista
```

Eta reduce

Found:

soloPares lista = crearFiltro even lista

Why Not:

```
[6]: soloPares [1..10]
```

```
[2,4,6,8,10]
```

```
[7]: -- EJERCICIO 4.3
```

```
enRango :: Int -> Int -> Int -> Bool
enRango minimo maximo numero =
  numero >= minimo && numero <= maximo
```

```
[8]: entre5y10 = enRango 5 10
```

```
[9]: entre5y10 7
```

```
True
```

```

-- use

[10]: entre5y10 3

False

[11]: -- EJERCICIO 4.4

potenciador :: Int -> (Int -> Int)
potenciador potencia numero = numero ^ potencia

[12]: potencia5 = potenciador 5

[13]: potencia5 7

16807

[14]: potencia5 2

32

```

```

[1]: import Data.List (sortOn)

-- EJERCICIO 5

type Estado = (Int, Int)
type Nodo = Estado
type Costo = Int
type Grafo = [(Nodo, [(Nodo, Costo)])]

capA :: Int
capA = 5

capB :: Int
capB = 3

-- Genera todos los estados alcanzables

vecinos :: Estado -> [(Estado, Costo)]
vecinos (a,b) =
  [ ((capA,b), 1)
    , ((a,capB), 1)
    , ((0,b), 1)
    , ((a,0), 1)
    , (transferirAB (a,b), 1)
    , (transferirBA (a,b), 1)
  ]

-- Verter de A hacia B

transferirAB :: Estado -> Estado
transferirAB (a,b) =
  let cantidad = min a (capB - b)
  in (a - cantidad, b + cantidad)

-- Verter de B hacia A

transferirBA :: Estado -> Estado
transferirBA (a,b) =
  let cantidad = min b (capA - a)
  in (a + cantidad, b - cantidad)

```

```
heuristica :: Estado -> Costo
heuristica (a,_) = abs (4 - a)
```

```
-- Algoritmo A*
```

```
aStar :: (Nodo -> [(Nodo, Costo)])
-> (Nodo -> Costo)
-> Nodo
-> Nodo
-> [(Nodo, Costo)]
```

```
aStar sucesores heuristica inicio meta =
  buscar [(inicio, 0)] []
```

```
where
```

```
  buscar [] _ = []
```

```
  buscar ((nodo,costo):cola) visitados
```

```
    | nodo == meta =
      [(nodo,costo)]
```

```
    | nodo `elem` visitados =
      buscar cola visitados
```

```
    | otherwise =
```

```
      let nuevos =
```

```
        [ (v, costo + c)
        | (v,c) <- sucesores nodo
        , v `notElem` visitados
        ]
```

```
        ordenados =
```

```
          sortOn (\(v,c) -> c + heuristica v)
            (cola ++ nuevos)
```

```
      in (nodo,costo) :
```

```
        buscar ordenados (nodo:visitados)
```

```
[2]: vecinos (0,0)
```

```
[[((5,0),1),((0,3),1),((0,0),1),((0,0),1),((0,0),1),((0,0),1)]]
```

```
[3]: aStar vecinos heuristica (0,0) (4,0)
```

```
[[((0,0),0),((5,0),1),((5,3),2),((2,3),2),((0,3),1),((3,0),2),((3,3),3),((2,0),3),((5,1),4),((0,2),4),((5,2),5),((4,3),6),((4,0),7)]]
```